EXPERIMENT 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Multi-Branch Vehicle Service and Repair Enterprise — Zoho Creator Case Study

Ananya P
241401010

1. Aim

To study, through a case study, the limitations of low-code platforms when they are used for a complex enterprise use case: a vehicle service and repair network that handles service requests across many branches, systems and user roles. The starting point is the *Service_Request_Form* built in Zoho Creator, which is then scaled up to an enterprise setting. The study also aims to recommend how the limitations can be reduced.

The case study shows where low-code gives clear benefit, where it begins to strain as complexity grows, and how a hybrid approach can keep the speed of low-code while avoiding its weak points.

2. Procedure

Step 1: Define the enterprise scenario — A realistic large-scale scenario was described: a multi-branch automobile service company that takes service and repair requests from owners through a single online form, with many employee roles, a legacy dealer management system or ERP, payment, insurer and parts-supplier APIs, strict data-privacy and financial rules, and high booking volumes.

Step 2: List the requirements — The scenario was broken into requirements such as scale, integration, custom logic, security, customisation and long-term maintenance (Section 3.2).

Step 3: Map the architecture — The scenario was drawn as an architecture diagram, and each integration was marked as well supported, possible with effort, or strained for a typical low-code platform (Section 3.1).

Step 4: Identify limitations — Each requirement was examined for limitations that are common across low-code platforms, with the impact on this case and a mitigation (Section 3.3).

Step 5: Show how limits appear with growth — A conceptual chart and an indicative suitability chart were prepared to show where low-code fits well and where it strains (Sections 3.4 and 3.5).

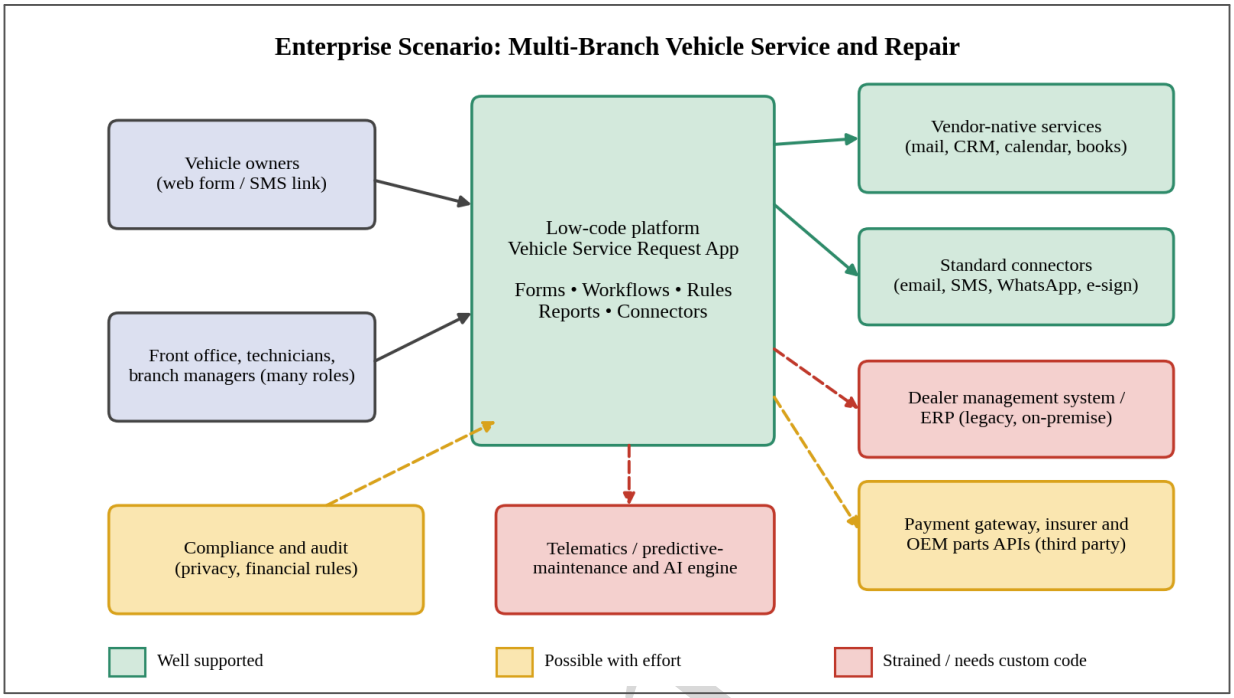
Step 6: Decide and mitigate — A decision flow was prepared to choose between pure low-code, hybrid and custom development (Section 3.6), and a hybrid architecture was proposed (Section 3.7).

Note: this is an analytical case study. Specific limits, such as execution time, record volume or API calls, differ between platforms and plans, so none are quoted here; the current documentation of the chosen platform should be checked. The charts are conceptual or indicative, not measured benchmarks.

241401010

3. Output

3.1 Enterprise scenario and integration map



The diagram shows the scenario around a generic low-code platform running the vehicle service request app. Vendor-native services and standard connectors are well supported (green). Payment, insurer and parts-supplier APIs and compliance needs are possible with effort (amber). The dealer management system or ERP and the telematics or AI engine for predictive maintenance are strained and usually need custom code (red).

3.2 Enterprise requirements

The requirements of the scenario are summarised below.

Requirement	Why it is complex
High volume and scale	Many branches, owners and service requests, with peaks on weekends, festival seasons and after recall notices.
Complex business rules	Branch-specific service bay and technician scheduling, estimate pricing, warranty and insurance rules that change often.
System integration	Legacy dealer system or ERP, payment and insurer APIs, OEM parts catalogue, identity and single sign-on.
Security and compliance	Owner personal data, financial regulations, audit trails, data residency.
Tailored experience	Different interfaces for owners, front office, technicians and branch managers, including mobile use in the workshop.
Long-term maintenance	Years of change by many teams, and the ability to change platform later.

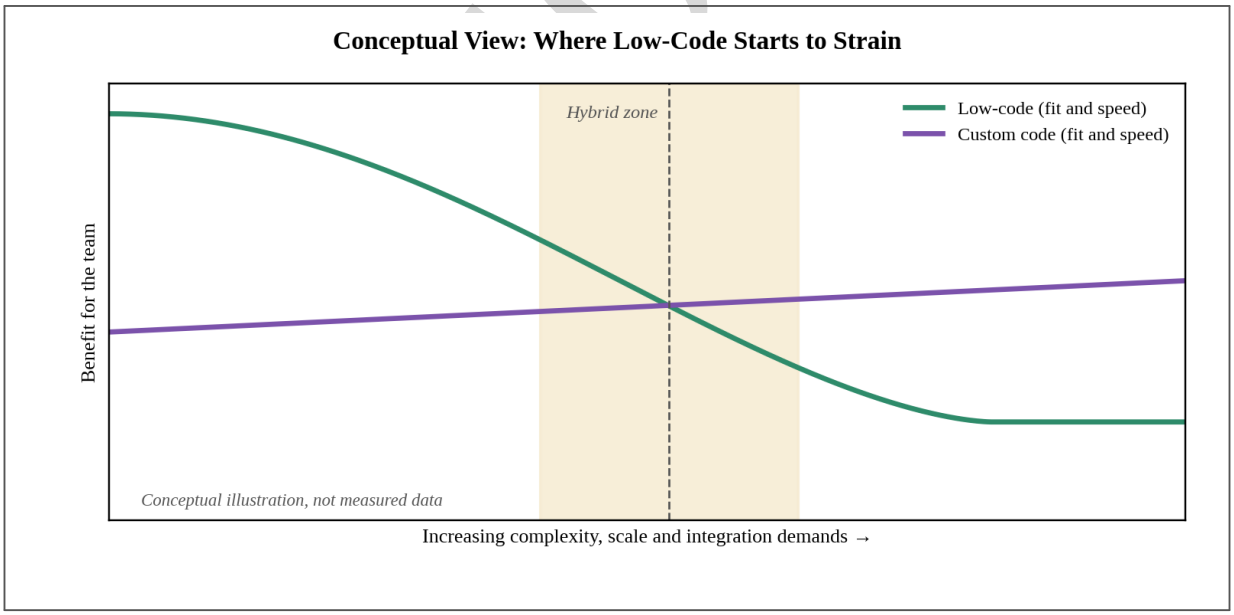
24401

3.3 Limitation analysis

The limitations below are common across low-code platforms, with their impact on this case and a possible mitigation.

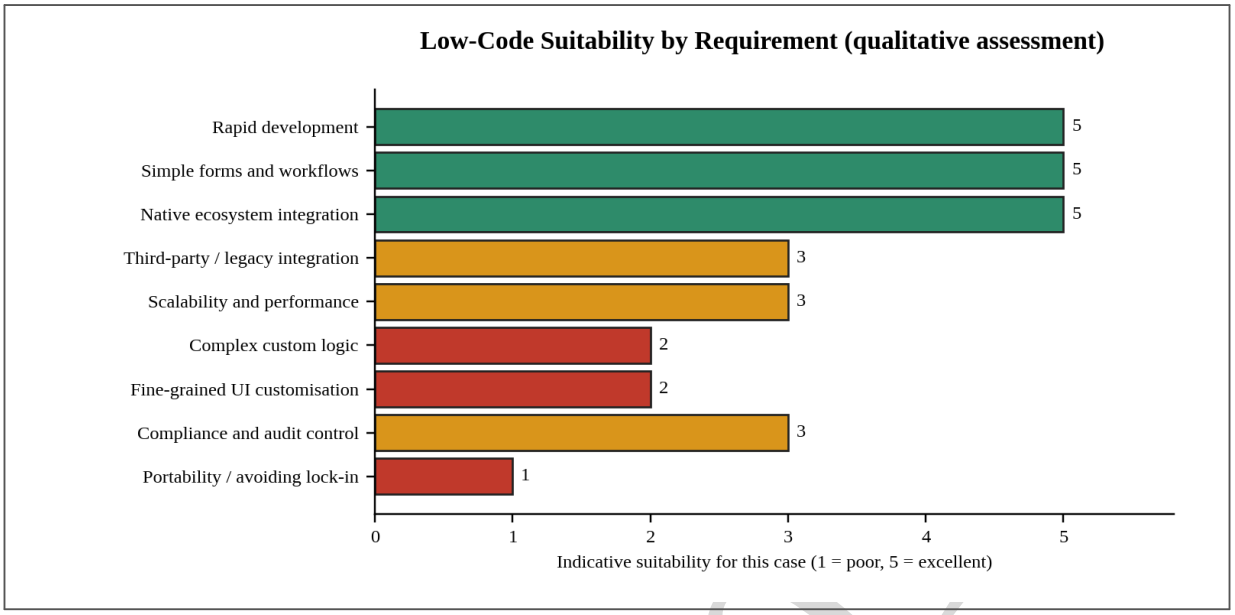
Area	Limitation of low-code	Impact on this case	Mitigation
Scalability and performance	Platform-managed runtime and shared resources; script execution and API usage are limited by platform and plan.	Peak booking days and large branch-wise reports may slow down or hit limits.	Efficient queries, avoid loops, move heavy jobs to external services.
Complex logic	Platform scripting suits business rules, not complex algorithms.	Technician and bay optimisation, dynamic estimate pricing and fraud checks on claims are hard to express.	Call a custom microservice through an API.
Integration	Native connectors cover popular services; legacy systems and industry standards need custom work.	Dealer system, ERP, OEM parts and insurer systems cannot be linked by simple connectors.	API gateway or middleware between the platform and legacy systems.
Customisation	Fixed UI components and platform conventions; form fields follow built-in field types.	Highly tailored interfaces, such as a vehicle inspection screen for technicians, are limited.	Custom components or a separate front end over the platform API.
Security and compliance	Hosting, data residency and audit depth depend on the vendor.	Owner contact and payment details may need more control than the platform gives.	Review certifications, role-based access, keep sensitive data in a controlled system.
Governance and sprawl	Easy building leads to many unmanaged apps.	Each branch may build its own copy of the form; duplicate or unsupported apps appear.	Central inventory, standards, centre of excellence.
Vendor lock-in	Apps, logic and workflows are tied to one platform and are hard to export.	A later move would mean rebuilding forms, reports and workflows.	Keep core logic in APIs and data exportable.

3.4 Where low-code starts to strain



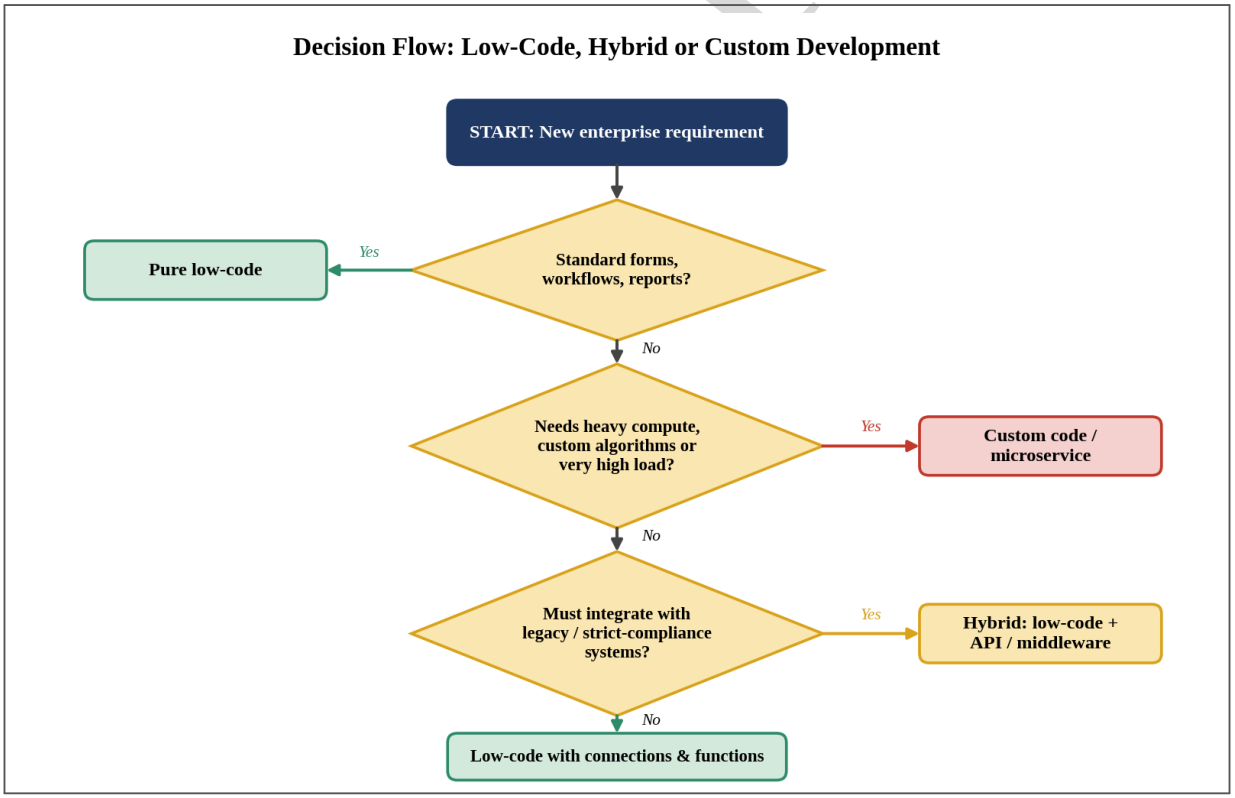
The conceptual chart shows that low-code gives a large benefit for simple needs such as the single service request form, but as complexity, scale and integration demands grow its advantage falls, while custom code stays steady. The hybrid zone marks where combining both is usually best. The chart is a conceptual illustration, not measured data.

3.5 Suitability of low-code by requirement



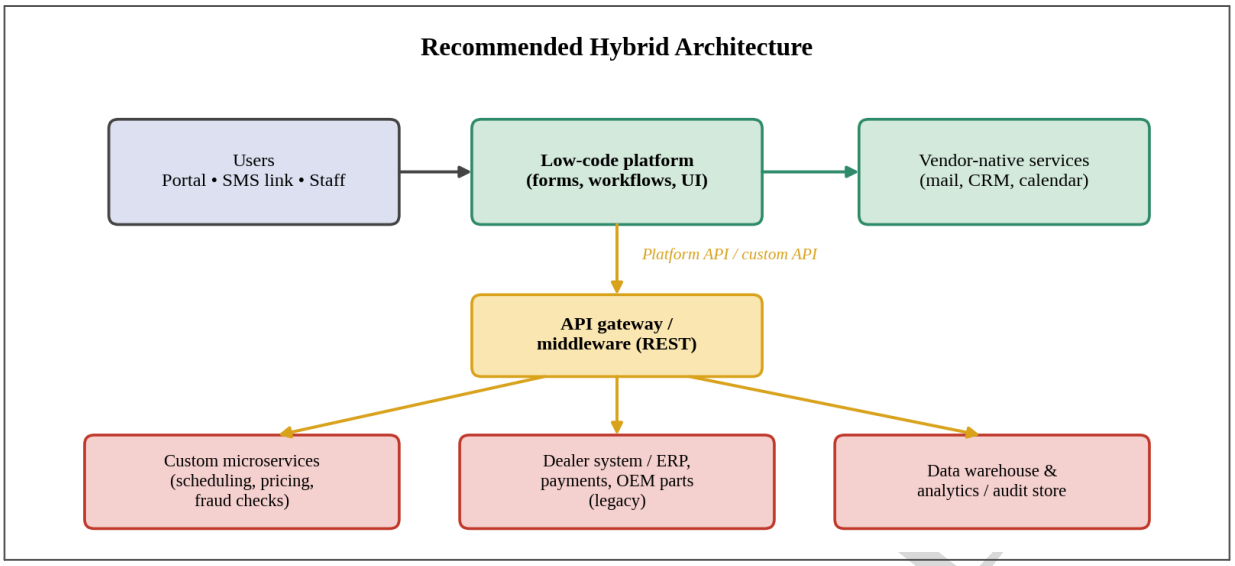
The chart rates how well low-code fits each requirement for this case. Rapid development, simple forms and workflows, and native ecosystem integration score highest, while custom logic, customisation and portability score lowest. The values are indicative scores from this analysis, not benchmark results.

3.6 Decision flow for choosing the approach



The decision flow asks whether the requirement is standard, whether it needs heavy computation or very high load, and whether it must integrate with legacy or strict-compliance systems. The answers lead to pure low-code, custom code, a hybrid design, or low-code with connectors and functions.

3.7 Recommended hybrid architecture



The low-code platform keeps the service request form, workflows, reports and the user interface, and uses vendor-native services. An API gateway or middleware layer links it to custom microservices, the legacy dealer system or ERP and payment systems, and a data warehouse for analytics and audit.

4. Final Result

The case study showed that low-code is very effective for the standard parts of an enterprise system — forms, workflows, notifications and ecosystem integrations — and gives fast development with little code, whichever platform is used. The Service_Request_Form built in Zoho Creator is a good example of this strength.

For a complex enterprise use case, the main limitations are scalability limits, complex custom logic, legacy and standards-based integration, restricted customisation, compliance control, governance sprawl and vendor lock-in. These were analysed in the limitation table, the conceptual chart and the suitability chart.

The recommended solution is a hybrid approach: keep low-code for forms, workflows and user interface, and add an API or middleware layer with custom services for heavy logic, legacy integration and analytics, under clear governance. No live testing or benchmark was carried out; the conclusions come from analysis of the case.

24AI01